TECHNICAL DOCUMENTATION

Service: dkim-milter

dkim-milter

Postfix milter in Go that DKIM-signs each message with the key of its `From:` domain. Keys per domain come from the table `dkim` in PostgreSQL or MySQL; private keys are stored encrypted with Vault Transit and decrypted into memory, so a sealed Vault blocks a restart, not signing. A message it cannot sign is accepted unsigned; the milter never holds mail.

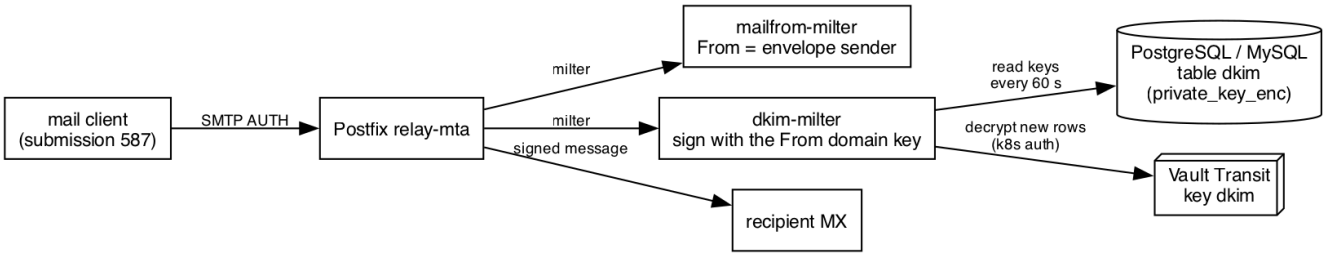


Figure 1: dkim-milter architecture

Stack

- Go 1.27 (`alpine:3.24` runtime image)
- `github.com/0kaba0hub/go-milter` v0.5.0 (milter protocol)
- `github.com/emersion/go-msgauth` v0.7.0 (DKIM, RSA and Ed25519)
- `github.com/jackc/pgx/v5`, `github.com/go-sql-driver/mysql` (key table)
- Vault Transit over HTTP, Kubernetes auth
- `log/slog` JSON logs, Prometheus metrics

Directory layout

```
dkim-milter/
|- app/go/          main.go config.go keys.go vault.go signer.go metrics.go, tests, Dockerfile
|- helm/            chart: Deployment, Service, ServiceAccount
|- helm_values/     values-sandbox.yaml (MySQL, keys in the clear), values-keptemail.yaml (Postgr
|- schema/          postgres.sql, mysql.sql
`- doc/             this document, architecture diagram
```

Components

Part	What
milter session	collects headers and body, signs at end of message, inserts `DKIM-Signature` at the top
key store	reads the key query, decrypts new rows, keeps the last good set on a failed reload
Vault client	Kubernetes login with the ServiceAccount token, one batch decrypt per load
metrics	`/healthz`, `/readyz` (keys loaded), `/metrics`

Environment variables

Variable	Default	What
`LISTEN_ADDR` / `METRICS_ADDR`	`0.0.0.0:8891` / `0.0.0.0:8081`	listeners
`DB_DRIVER` / `DB_DSN`	`postgres` / —	database
`DKIM_SQL_QUERY`	`SELECT domain, selector, private_key_enc FROM dkim`	key query
`KEY_DECRYPT`	`vault`	`vault` or `none`
`KEY_RELOAD_SECONDS`	`60`	`0` loads once
`VAULT_ADDR` / `VAULT_ROLE`	—	required with `vault`
`VAULT_AUTH_PATH` / `VAULT_TRANSIT_PATH` / `VAULT_TRANSIT_KEY`	`kubernetes` / `transit` / `dkim`	mounts and key
`VAULT_SA_TOKEN_FILE` / `VAULT_CACERT`	SA token path / —	login token, CA
`LOG_LEVEL`	—	`debug`

Deploy

ArgoCD application `argocd-app.yaml` (chart `helm/`, values from `helm_values/`). The DSN comes from a Secret (`db.dsnSecret`); with `keyDecrypt: vault` the chart mounts the ServiceAccount token for the Vault role. Postfix: `smtpd_milters` after `mailfrom-milter`, `non_smtpd_milters`, `milter_default_action = tempfail`.

Logging

JSON on stdout, one line per message: `dkim: signed` with domain, selector and queue id, or `dkim: not signed` with the reason (no From domain, no key, error). Key loads and reloads, Vault failures and unusable keys are logged with the domain they concern.